

YORU

Identidad digital para líneas telefónicas protegida con
criptografía de curva elíptica **ECDSA P-256**

Guía de preguntas y respuestas

Feria de proyectos

Documento de apoyo para la defensa técnica ante maestros de matemáticas, criptografía y programación. Incluye capturas de código con su ruta de archivo, basadas en el código real del proyecto.

Contenido

- 1. Preguntas sobre el código (programación)
- 2. El algoritmo ECDSA y sus procesos
- 3. Dónde y cómo se guardan los datos y su seguridad

1 · Preguntas sobre el código

Arquitectura, comunicación entre servicios y decisiones de implementación.

P: ¿Por qué microservicios y no un monolito?

R: Para aislar responsabilidades y datos: cada servicio tiene su propia base de datos (database-per-service). El servicio que guarda las llaves (PKI) está separado del que guarda las cuentas (auth); aunque comprometieran uno, no se llevan todo. Además se pueden desplegar y escalar por separado.

P: ¿Cómo se comunican los 5 servicios entre sí?

R: De dos formas: HTTP/REST para llamadas síncronas (p. ej. auth llama a pki para verificar una firma) y **RabbitMQ** (exchange tipo `topic` y `yoru.events`) para eventos asíncronos como el kill switch y las notificaciones.

P: ¿Qué pasa si RabbitMQ o un servicio se cae?

R: Los clientes del bus se reconectan solos cada 3 s y vuelven a registrar sus suscripciones, así no se pierden eventos en silencio si un servicio arranca antes que el broker o el broker se reinicia.

P: ¿Cómo suben los archivos pesados (INE en PDF, selfie)?

R: Con **presigned URLs**: el backend firma una URL temporal (5 min) y el navegador sube el archivo **directo a MinIO**, sin pasar por nuestros servidores. Ahorra ancho de banda y escala mejor. El servicio nunca toca los bytes del archivo, solo gestiona la referencia.

```
ruta: yoru-backend/services/kyc-service/src/s3.js

export async function presignPut({ key, contentType }) {
  return getSignedUrl(
    s3,
    new PutObjectCommand({ Bucket: BUCKET, Key: key, ContentType: contentType }),
    { expiresIn: 300 } // la URL caduca en 5 minutos
  );
}
```

P: ¿Cómo evitan datos huérfanos si el registro falla a medio camino?

R: El registro es **diferido**: nada se persiste en KYC/PKI/Telecom hasta que el usuario verifica su correo. Si algo falla, se hace rollback con el evento `auth.user_deleted` que los servicios consumen; además un proceso de limpieza borra los borradores no confirmados a los 30 min.

P: Acaban de pedir limitar las líneas. ¿Cómo se hizo?

R: Con una validación en el backend (la fuente de verdad), aplicada **antes de enviar el SMS** y otra vez **antes de vincular** (por si se arrancan dos flujos en paralelo). El frontend además oculta el botón al llegar al tope.

ruta: yoru-backend/services/auth-service/src/routes/auth.js

```
const MAX_LINEAS = 10; // máximo de líneas por usuario EN TOTAL

async function contarLineas(userId) {
  const r = await fetch(`${TELECOM_URL}/api/telecom/lineas/by-user/${userId}`);
  if (!r.ok) return null;
  const d = await r.json();
  return Array.isArray(d.lineas) ? d.lineas.length : 0;
}

// en /lineas/verificar/iniciar y en /lineas/agregar:
const totalLineas = await contarLineas(userId);
if (totalLineas !== null && totalLineas >= MAX_LINEAS) {
  return reply.code(409).send({ ok: false,
    error: 'Alcanzaste el máximo de 10 líneas por cuenta.' });
}
```

P: ¿Web Crypto API en el navegador es seguro? ¿No es JavaScript “casero”?

R: No es casero: es la API criptográfica nativa estandarizada por la W3C, implementada por el motor del navegador. Permite además marcar las llaves como **no exportables**, justo lo que aprovechamos para que la llave privada no se pueda leer desde el código de la página.

P: ¿Cuál es el stack?

R: Frontend: React + Vite + Tailwind. Backend: Node.js + Fastify + Prisma. Datos: PostgreSQL (uno por servicio), RabbitMQ y MinIO. Todo orquestado con Docker Compose.

2 · El algoritmo ECDSA y sus procesos

ECDSA = Elliptic Curve Digital Signature Algorithm, sobre la curva P-256 (secp256r1).

Las matemáticas en breve. La curva P-256 es el conjunto de puntos (x, y) que cumplen $y^2 = x^3 - 3x + b \pmod{p}$ con p primo de 256 bits. Sus puntos forman un grupo cíclico de orden n con generador G . La operación base es la multiplicación escalar $k \cdot G$. La seguridad viene del **problema del logaritmo discreto elíptico (ECDLP)**: dado $Q = d \cdot G$ es inviable recuperar d . Da ~ 128 bits de seguridad con llaves de solo 256 bits.

Fase 1 — Generación del par de llaves

●●● ruta: registro/registro/src/screens/Generacion.jsx

```
// Genera el par ECDSA P-256 en el navegador (Web Crypto API)
const keyPair = await window.crypto.subtle.generateKey(
  { name: 'ECDSA', namedCurve: 'P-256' },
  true, // exportable: para guardarlo en el .pem
  ['sign', 'verify'],
);
const publicKeyPem = await publicKeyToPem(keyPair.publicKey); // SPKI
const privateKeyPem = await privateKeyToPem(keyPair.privateKey); // PKCS8
```

Matemática: se elige d aleatorio en $[1, n-1]$ (privada) y se calcula $Q = d \cdot G$ (pública). Solo Q se envía al servidor.

Fase 2 — Firma del reto

●●● ruta: registro/registro/src/api.js · signWithPrivateKey()

```
const dataBytes = new TextEncoder().encode(payload); // el reto (nonce)
const sigBuffer = await crypto.subtle.sign(
  { name: 'ECDSA', hash: 'SHA-256' }, // hash + firma
  privateKey,
  dataBytes,
);
return btoa(...); // firma (r, s) en base64 (formato IEEE P1363)
```

Matemática: $e = \text{SHA-256}(m)$ · k aleatorio secreto · $r = (k \cdot G)_x \pmod{n}$ · $s = k^{-1}(e + r \cdot d) \pmod{n}$ · la firma es el par (r, s) .

Fase 3 — Verificación (servidor)

ruta: yoru-backend/services/pki-service/src/routes/pki.js

```
const publicKey = await importPublicKeyPem(key.publicKeyPem); // SPKI
const sigBytes = Buffer.from(signatureB64, 'base64');
const dataBytes = new TextEncoder().encode(payload);

valida = await crypto.subtle.verify(
  { name: 'ECDSA', hash: 'SHA-256' },
  publicKey, sigBytes, dataBytes
);
```

Matemática: $w = s^{-1} \cdot u_1 = e \cdot w \cdot u_2 = r \cdot w \cdot (x_1, y_1) = u_1 \cdot G + u_2 \cdot Q$ · válida si $r \equiv x_1 \pmod{n}$. El servidor solo necesita la llave pública.

El reto-respuesta: nonce de un solo uso (anti-replay)

ruta: yoru-backend/services/auth-service/src/routes/auth.js

```
const nonce = crypto.randomUUID() + '.' + Date.now();
const expiresAt = new Date(Date.now() + 5 * 60 * 1000); // 5 min
await prisma.challenge.create({
  data: { userId, nonce, proposito: 'recovery', expiresAt },
});
// al verificar la firma:
if (challenge.usado) ... // reto de un solo uso
if (challenge.expiresAt < new Date()) ... // y con expiracion
```

Preguntas frecuentes sobre ECDSA

P: ¿Por qué 256 bits bastan si RSA necesita miles?

R: Los mejores ataques contra ECDLP son de orden $\sqrt{n}$, así que 256 bits dan ~128 bits de seguridad. RSA depende de factorización, con algoritmos sub-exponenciales mejores, por eso necesita ~3072 bits para la misma seguridad. Resultado: llaves más pequeñas y firmas más rápidas.

P: ¿Qué papel juega el número aleatorio k?

R: Es efímero y secreto por firma. Si k se repite o se predice, se puede despejar la llave privada d (fue el famoso fallo de la PS3 de Sony). Por eso lo genera un RNG criptográfico; en nuestro caso lo provee Web Crypto.

P: ¿Por qué se firma el hash y no el mensaje completo?

R: Por eficiencia y porque ECDSA opera con un entero acotado por n. SHA-256 da un resumen de tamaño fijo; cambiar un bit del mensaje cambia todo el hash (efecto avalancha).

P: ¿Reto-respuesta en vez de enviar una firma fija?

R: Una firma fija sería reutilizable (replay). El servidor emite un nonce único; la firma vale solo para ese reto, es de un solo uso y caduca en 5 min. Nunca se transmite la llave privada.

P: ¿Qué formato de firma usan?

R: IEEE P1363 (raw r||s, 64 bytes), que es lo que produce y consume Web Crypto API; lo transmitimos en base64. No es DER/ASN.1 como OpenSSL por defecto.

3 · Dónde y cómo se guardan los datos (y su seguridad)

Cada microservicio tiene su propia base de datos; los archivos pesados viven en almacenamiento de objetos; la llave privada nunca toca el servidor.

auth-db PostgreSQL · :5433 cuentas, contraseñas (hash), sesiones, retos	pki-db PostgreSQL · :5434 llaves públicas, bitácora de firmas	kyc-db PostgreSQL · :5435 referencias INE/selfie, estado KYC
telecom-db PostgreSQL · :5436 líneas y eventos de línea	MinIO Object storage · :9000 archivos: PDF de INE y selfie	RabbitMQ Bus · :5672 eventos (efímeros)
		Archivo .pem en el dispositivo del usuario llave privada — nunca en el servidor

Dato	Dónde se guarda	Cómo / qué seguridad tiene
Cuenta (CURP, teléfono, correo)	auth-db (PostgreSQL :5433)	Aislada por servicio; CURP/teléfono/correo únicos.
Contraseña	auth-db	Nunca en claro: scrypt + salt aleatorio de 16 bytes.
Llave pública	pki-db (:5434)	Solo la pública; permite verificar, no firmar.
Llave privada	Archivo .pem del usuario	Nunca llega al servidor; se importa como no exportable.
Firmas (auditoría)	pki-db	Hash del payload + resultado (válida/ inválida).
INE (PDF) y selfie	MinIO, bucket kyc-documents (:9000)	El archivo va a object storage; en la BD solo la referencia.
Estado KYC + refs	kyc-db (:5435)	Guarda referencias, no los archivos.
Líneas telefónicas	telecom-db (:5436)	Estado (activa, bloqueada, kill_switched) + eventos.
JWT de sesión	Memoria del navegador	No en localStorage (mitiga XSS); expira en 1 h.
Eventos del sistema	RabbitMQ (:5672)	Bus de mensajería; no es almacén permanente.

Contraseñas: scrypt + salt + comparación en tiempo constante

```
ruta: yoru-backend/services/auth-service/src/password.js

export function hashPassword(plain) {
  const salt = crypto.randomBytes(16).toString('hex'); // salt unico
  const hash = crypto.scryptSync(plain, salt, 64).toString('hex');
  return { hash, salt };
}

export function verifyPassword(plain, hash, salt) {
  const test = crypto.scryptSync(plain, salt, 64);
  const stored = Buffer.from(hash, 'hex');
  return crypto.timingSafeEqual(stored, test); // tiempo constante
}
```

- **script**: función de derivación lenta y con uso intensivo de memoria → encarece la fuerza bruta y los ataques con GPU.
- **Salt único** por usuario: dos personas con la misma contraseña tienen hashes distintos (mata las *rainbow tables*).
- **timingSafeEqual**: compara sin filtrar información por el tiempo de comparación (anti canal lateral).

La llave privada nunca sale del dispositivo

```

ruta: registro/registro/src/api.js · importPrivateKeyFromPem()

return crypto.subtle.importKey(
  'pkcs8', der,
  { name: 'ECDSA', namedCurve: 'P-256' },
  false,          // extractable:false -> no se puede re-exportar
  ['sign'],       // solo permiso de FIRMAR
);

```

R: El archivo .pem tiene 3 bloques: YORU IDENTITY (metadata), PUBLIC KEY (SPKI) y PRIVATE KEY (PKCS8). Al cargarlo, la privada se importa como no exportable y solo para firmar; por la red solo viaja la firma. Esto elimina la categoría de ataque “rompo el servidor y me llevo las llaves”.

Qué guarda exactamente la base de datos de cuentas

```

ruta: yoru-backend/services/auth-service/prisma/schema.prisma

model User {
  id          String   @id @default(uuid())
  curp        String   @unique
  telefono    String   @unique
  email       String?  @unique
  passwordHash String?  // script
  passwordSalt String?  // salt aleatorio
  committed   Boolean  @default(false) // alta real solo si true
  emailVerified Boolean @default(false)
  // ...codigos temporales de verificacion, recuperacion y SMS...
}

```

Qué guarda PKI (solo lo público) + bitácora de firmas

```

ruta: yoru-backend/services/pki-service/prisma/schema.prisma

model PublicKey {
  id          String   @id @default(uuid())
  userId      String
  publicKeyPem String   @db.Text // SOLO la llave publica
  curva       String   @default("P-256")
  revocada    Boolean  @default(false)
}

model Signature { // auditoria de cada verificacion
  payloadHash String // hash SHA-256 del payload
  resultado   String // valida | invalida
}

```

Almacenamiento de archivos (MinIO) y su acceso

ruta: yoru-backend/services/kyc-service/src/s3.js

```
export const s3 = new S3Client({
  endpoint: 'http://localhost:9000', // MinIO (S3-compatible)
  credentials: { accessKeyId: ..., secretAccessKey: ... },
  forcePathStyle: true,
});
// El INE (PDF) y la selfie se suben con presigned URL directo a MinIO;
// en la base de datos KYC solo se guarda la referencia (key), no el archivo.
```

Resumen de seguridad

- Llave privada client-side, no exportable, nunca viaja por la red.
- Reto–respuesta con nonce de un solo uso (anti-replay), expira en 5 min.
- Firmas ECDSA P-256 verificadas en el servidor, con bitácora de cada intento.
- Contraseñas con scrypt + salt y comparación en tiempo constante.
- JWT HS256 (1 h) con autorización por dueño: no puedes tocar recursos ajenos aunque tengas token.
- Kill switch automático ante intento de registrar una segunda llave.
- Database-per-service + comunicación por eventos + CORS restringido a la app.

Honestidad técnica (limitaciones de demo). KYC se auto-aprueba sin matching facial real; no hay API Gateway ni rate limiting global; los secretos de los .env son de demo (irían a un secret manager); faltan tests automatizados. La criptografía (ECDSA, scrypt, JWT) sí es estándar y real.